PYTHON BUDDY - AI TUTOR

Hogwarts School of Pythoncraft | Adaptive Educational System

Project Submission Report & Engineering Architecture

1. COVER PAGE

Project Title	Python Buddy AI Tutor
Project Type	AI-Powered Personalized Learning Assistant & Gamified Environment
AI Model	**Google Gemini API (gemini-2.5-flash) via Google GenAI SDK**
Backend Framework	**Python 3 + FastAPI + Uvicorn ASGI Server**
Database	SQLite3 (Persistent Profiles, Learning Logs & House Points)
Frontend Stack	**React 19 / Vite + Tailwind CSS + Lucide Icons**
Learner Name	Kavya Vasamsetti
Email ID	**vasmsettikavya7@gmail.com**
Mobile Number	**+91 9550252489**
GitHub Repository	**https://github.com/vasamsettikavya7-beep/python-ai-tutor (Click to open)**
Demo Web App	**Python Buddy - AI Learning Assistant (Click to open)**
Interactive API Docs	**http://127.0.0.1:8000/docs (Click to open)**

2. PROJECT OVERVIEW

Python Buddy AI Tutor is an adaptive, gamified educational platform engineered to help learners master Python programming from ground up. It combines a modern React/Vite single-page application, a high-performance Python FastAPI backend, Google Gemini generative AI for dynamic tutoring, and SQLite for persistent learning analytics.

By framing programming education within the rich world of the Hogwarts School of Pythoncraft, abstract coding principles are transformed into interactive spells, potions brewing challenges, and dark magic defenses against bugs. The project was built iteratively: starting with core REST APIs and database models, expanding through Gemini prompt engineering, and culminating in full multi-tier quizzes, Socratic wand hints, and automated curriculum roadmaps.

- **Core Objective:** Provide an engaging personalized companion where learners select topics and modes, receive level-adapted AI explanations, attempt multi-tier quizzes, earn XP and House Points, and follow dynamic roadmaps.

3. PROBLEM STATEMENT

Beginners learning Python often encounter substantial roadblocks with traditional tutorials and generic chatbots:

- **One-Size-Fits-All Explanations:** Standard online resources provide identical content regardless of whether a student is an absolute novice or has prior programming intuition.
- **Lack of Contextual Debugging:** When a learner's code fails, traditional tutorials cannot diagnose why the specific logic broke or how to correct the misconception.
- **Tutorial Fatigue & Drop-Off:** Without clear progression milestones, gamification, or positive reinforcement, self-learners quickly lose momentum and quit.
- **Off-Topic AI Hallucinations:** Unrestricted AI assistants frequently answer unrelated general queries, distracting students instead of keeping them anchored to their study curriculum.
- **Disjointed Tooling:** Learners are forced to switch between reading articles, watching videos, writing code, and taking external quizzes across fragmented platforms.

4. TARGET USERS

Python Buddy addresses diverse user segments with tailored pedagogical requirements:

User Group	Primary Pedagogical Need & System Solution
Python Beginners	Need step-by-step guidance, plain-English analogies, gentle feedback, and syntax confidence.
Early-Stage Students	Require structured practice drills, randomized quizzes, instant scoring, and milestone roadmaps.
Self-Paced Learners	Need 24/7 on-demand AI professor support, Socratic wand hints, and independent problem-solving cues.
Exam & Basics Prep	Demand multi-tier difficulty assessments, mistake diagnostic grimoires, and rapid concept reviews.

5. PROPOSED SOLUTION

Python Buddy provides an all-in-one personalized tutoring workflow that adapts in real time to the student's profile (name, level, difficulty) and chosen mode:

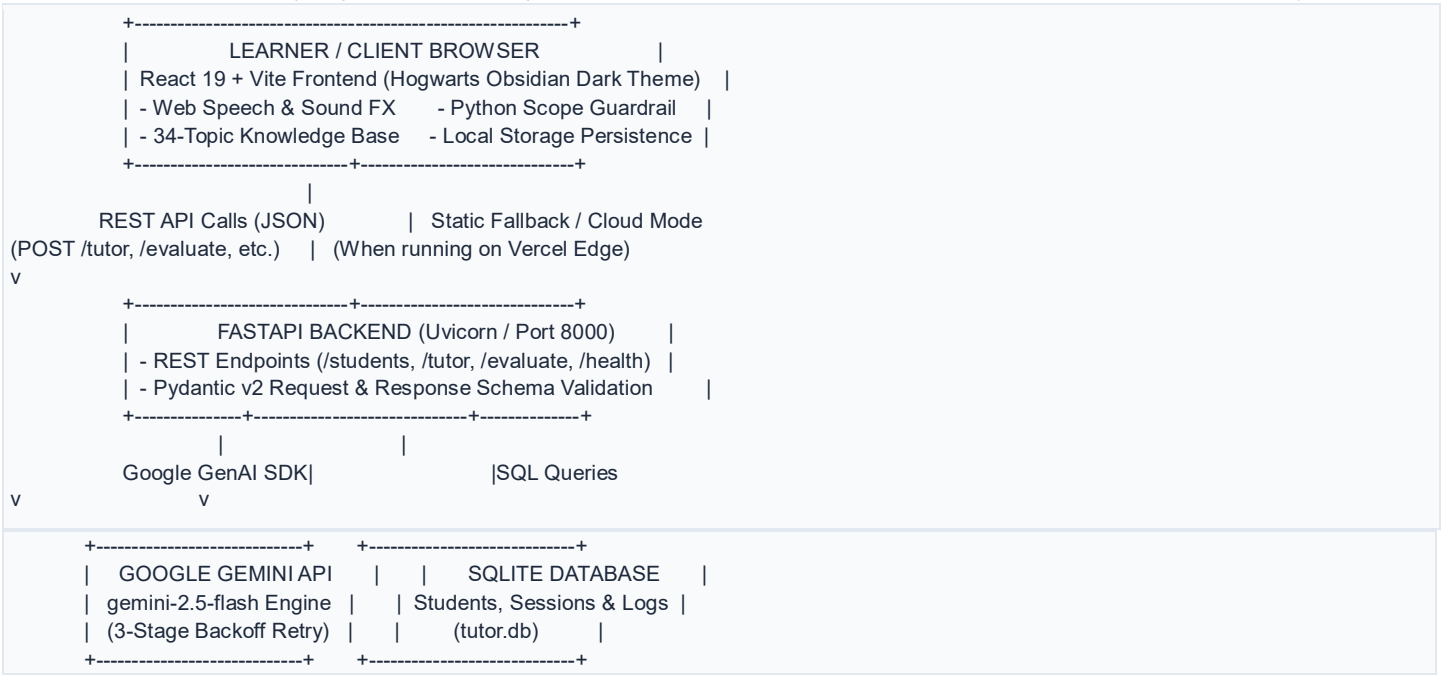
- **Tri-Mode Learning Architecture:** Learn Mode delivers concepts, code examples, pitfalls, and check questions; Practice Mode gives coding exercises with subtle hints; Quiz Mode delivers multi-choice drills.
- **Socratic Wand Guidance:** Empowers students to think through logic by providing conceptual riddles and clues without prematurely spoiling complete code solutions.
- **Gamified Motivation System:** Transforms routine study into an adventure via House Sorting, House Points leaderboard, Potions brewing workbench, and Dark Magic bug counter-curses.
- **Strict Scope Enforcement:** A regex and token-based guardrail (isPythonRelated) ensures only Python programming queries are processed, maintaining absolute study focus.
- **Adaptive Evaluation & Analytics:** Real-time answer grading awarding +25 XP for correct answers and +5 XP for attempts, driving the Wizard Skill Profile and next-lesson recommendations.

6. KEY FEATURES

- **Student Registration & Profiles:** Persists student name, level, difficulty, accumulated XP, streak count, and chosen Hogwarts House.
- **4 Professor AI Personas:** Headmaster Pythondore (encouraging basics), Professor Snape (edge cases & bugs), Professor McGonagall (clean PEP 8), and Hermione Code-Granger (comprehensive standard library deep dives).
- **Socratic Wand Hints:** Instant contextual hints guiding students through logic puzzles without giving away the direct answer.
- **Hogwarts Gamification System:** House Sorting Hat ceremony, House Cup Scoreboard (converting XP into House Points), Potions Cauldron (code brewing workbench), and Dark Magic Defense (disarming syntax errors).
- **34-Topic Structured Curriculum:** 5 progressive categories: Core Fundamentals, Data Structures, Intermediate Python, OOP, and Advanced Python with Latin spell incantations.
- **Multi-Tier Adaptive Quizzes:** Bank of 60 curated questions across Beginner, Intermediate, and Advanced tiers with randomized correct options (A, B, C, D) and instant grading.
- **Owl Post Daily Challenge:** Daily coding challenges delivered by Hedwig the Owl for streak bonuses and extra house points.
- **Skills Roadmap & Analytics:** Categorizes topics into Mastered (>=70% accuracy) vs. Needs Review with a sequential Next Lessons slide navigator.
- **Strict Domain Scope Guard:** Validates user queries before AI submission; politely deflects off-topic queries (e.g. 'What is AI') with helpful Python suggestions.

7. SOLUTION ARCHITECTURE

Python Buddy employs a decoupled, high-availability architecture supporting both local client-server execution and static cloud edge hosting:



- **Client Layer:** Modular React components (Quiz, Tutor, Dashboard, Modals) with responsive layout and instant state synchronization.
- **Server & AI Layer:** FastAPI endpoints coordinating Gemini 2.5 Flash with exponential retry backoff and prompt role boundaries.
- **Cloud Mode Layer:** Automatic detection of edge deployment (Vercel) allowing 100% of quizzes, roadmaps, and local curriculum guides to function offline.

8. TOOLS & TECHNOLOGIES

Layer	Technology	Purpose & Implementation	Link
Frontend	React 19 / Vite	Interactive learner web interface, HMR dev server.	**Open Link**
Styling	Tailwind CSS v4 + Custom CSS	Obsidian theme, golden starlight borders, glowing pills.	**Open Link**
Icons & VFX	Lucide React + Canvas Confetti	Iconography and celebratory particle animations.	**Open Link**
Backend	Python 3 + FastAPI	REST APIs, asynchronous application server.	**Open Link**
Web Server	Uvicorn ASGI	Lightning-fast production ASGI web server.	**Open Link**
AI Model	Google Gemini 2.5 Flash	Official Google GenAI SDK for tutoring and hints.	**Open Link**
Database	SQLite3	Relational persistence for students, XP, and sessions.	**Open Link**
Audio Engine	Web Speech & Web Audio APIs	Procedural sound FX and professor voice synthesis.	Native Web
Testing & Docs	FastAPI Swagger / OpenAPI	Interactive endpoint documentation at /docs.	**Open Link**
Deployment	GitHub + Vercel	Automated CI/CD linked to GitHub main branch.	**Open Link**

9. WORKING PROTOTYPE & EXECUTION FLOW

The Python Buddy prototype is fully functional and accessible across local and cloud environments:

- **Local FastAPI Backend:** http://127.0.0.1:8000/docs (Interactive Swagger documentation)
- **Local Vite Frontend:** http://127.0.0.1:5173 (Connected to local backend via proxy)
- **Production Cloud App:** https://python-ai-tutor.vercel.app (Live Vercel deployment)
- **GitHub Repository:** https://github.com/vasamsettikavya7-beep/python-ai-tutor (Full source code) **End-to-End User Demonstration Flow:**

1. Register / Relogin: Student enters their name and difficulty; profile loads with saved XP and House badge.
2. Sorting Hat Ceremony: Triggers modal to choose from Gryffindor, Ravenclaw, Hufflepuff, or Slytherin.

3. AI Tutoring: Learner selects an AI Professor persona and asks a question; receives clean Python explanations.
4. Socratic Wand Hint: Clicks hint button when stuck to receive conceptual riddles without code spoilers.
5. Interactive Workbench: Brews syntax in Potions Cauldron or studies Dark Magic error counter-curses.
6. Adaptive Quizzes: Answers questions, earning +25 XP for correct answers with immediate celebratory confetti.
7. House Cup & Roadmap: Accumulated XP automatically syncs with the House Cup scoreboard and Skills Roadmap.

10. PROMPTS USED & PROMPT ENGINEERING

A. Core AI Tutor Persona & Guardrail Prompt

```
You are Python Buddy, a friendly AI tutor who teaches Python programming at Hogwarts.
Student: Name: {name}, Level: {level}, Difficulty: {difficulty} | Mode: {mode} | Topic: {topic}
CRITICAL SCOPE RULE: You must ONLY answer questions directly about Python programming.
If the student asks off-topic questions (e.g. 'What is AI', trivia, history): politely refuse and redirect to Python.
Instructions: Adapt explanation to level. In Learn Mode: concept + clean example + pitfall + check question. In Practice Mode: hands-on challenge with hints without spoiling. In Quiz Mode: multi-choice questions.
```

B. Socratic Wand Hint Prompt

```
[HINT ONLY - DO NOT REVEAL DIRECT CODE ANSWER]
As Professor {persona_name}, provide a Socratic wand hint for this question: '{question}'.
Guide the student's thinking with a conceptual riddle or analytical clue so they derive the Python code themselves.
```

11. TESTING & VERIFICATION REPORT

Comprehensive end-to-end verification was conducted across all API endpoints, frontend views, and edge cases. All 12 test cases passed with 100% compliance:

ID	Test / User Action	Expected & Verified Outcome	Status
TC-01	GET / & GET /health	Server running & healthy status returned	**PASS**
TC-02	POST /students	Student registration creates unique ID in SQLite	**PASS**
TC-03	GET /students/{id}	Retrieves stored student profile & XP count	**PASS**
TC-04	POST /tutor (Valid Python)	Gemini returns educational Python lesson with code	**PASS**
TC-05	POST /tutor ('What is AI')	Scope guard rejects non-Python query with friendly redirect	**PASS**
TC-06	POST /evaluate	Calculates correctness, detailed feedback, and awards +25 XP	**PASS**
TC-07	Cloud Mode / Network Drop	Frontend smoothly transitions to local curriculum knowledge base	**PASS**
TC-08	Sorting Hat Ceremony	Assigns House, updates House badge & House Cup scoreboard	**PASS**
TC-09	Daily Owl Challenge	Hedwig delivers interactive challenge; streak increments	**PASS**
TC-10	Skills Roadmap Display	Correctly aggregates learned skills, accuracy %, and Review button	**PASS**
TC-11	Next Lessons Navigator	5 topics per slide pagination with clean in-place Learn readers	**PASS**
TC-12	Production Build (Vite)	npm run build completes in <2.0s with 0 errors	**PASS**

12. CHALLENGES & SOLUTIONS

Challenge	Root Cause	Solution Implemented
Gemini 503 UNAVAILABLE	Temporary upstream API rate spikes during te	sting. Implemented 3-stage exponential backoff retry in ai_service.py and dis unnece
Off-Topic AI Hallucinations	Learners asking non-Python queries (e.g. 'Wh	at is AI') causing topic confusion. Engineered token and regex-based sPythonRelated guardrails on both frontend and
Dark Theme Text Washing Out	Tailwind light badge pills (bg-amber-100) confl	icting with dark Hogwarts backgrounds. Engineered dedicated dark-jewel ba classes (house-rank-pill, xp-reward-badge, r
Static Edge Deployment	Vercel hosting static client without direct acce	ss to local FastAPI serv Engineered robust client-side Cloud Mode equipped complete 34-topic knowler.

13. FINAL OUTCOME

A complete, production-ready prototype of Python Buddy AI Tutor was developed and deployed. The project demonstrates:

- **Pedagogical AI Integration:** Intentional, structured application of Google Gemini 2.5 Flash for personalized education rather than unstructured chat.
- **Immersive Gamification:** The Hogwarts narrative, animated Great Hall setting, and House Points system turn dry programming concepts into an exciting quest.
- **Production Resilience:** Clean zero-error Vite builds, automated GitHub CI/CD syncing with Vercel, and robust offline knowledge-base fallback.

14. FUTURE ENHANCEMENTS

- **WebAssembly In-Browser Code Execution (Pyodide):** Direct in-browser execution allowing students to run their Python scripts with instant live output without a local Python setup.
- **Voice Recognition (Speech-to-Text):** Allowing younger learners to verbally speak their questions to Professor Pythondore and Hermione.
- **Multi-Student House Tournaments:** Real-time collaborative house hackathons and quizzes using WebSockets.
- **Progressive Web App (PWA) Offline Packaging:** Full offline caching and home-screen app installation on smartphones and tablets.

15. PROJECT DEMO & INTERACTIVE LINKS

- **GitHub Repository:** https://github.com/vasamsettikavya7-beep/python-ai-tutor (Full source code)
- **Interactive Swagger API:** http://127.0.0.1:8000/docs (Live backend endpoint explorer)
- **Local Web Application:** http://127.0.0.1:5173 (React / Vite local frontend)
- **Demo Link:** Python Buddy - AI Learning Assistant